

CL-2005: Database Systems

Lab # 5: Introduction to SQL

Objective:

To explore DML commands

Scope:

The student shall know the following

- Workaround SQL Server.
- SQL Practice.
- Hands-on experience on the above-mentioned concepts.

Discussion

SQL Commands

1. Data Insertion:

1.1. Inserting Data

- **Command:** INSERT INTO tableName (column1, column2, ...) VALUES (value1, value2, ...);
- **Purpose:** Adds new rows/tuples/records to a table.
- **Note:** If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table.
- INSERT INTO tableName VALUES (value1, value2, value3, ...);

Example:

```
-- Insert data into Customers Table
INSERT INTO Customers (customerName, contactEmail)
VALUES
    ('John Doe', 'john.doe@example.com'),
    ('Jane Smith', 'jane.smith@example.com');
```

```
-- Insert data into Sales Table
INSERT INTO Sales (productName, saleDate, amount, isCompleted, customerId)
VALUES
    ('Laptop', '2024-09-01', 1200.00, 1, 1),
    ('Smartphone', '2024-09-02', 800.00, 0, 2);
```

1.2. Updating Data

- **Command:** UPDATE tableName SET column1 = value1 WHERE condition;
- **Purpose:** To modify existing records in a table.

Example

```
UPDATE Students  
SET LastName = 'Smith'  
WHERE StudentID = 1;
```

1.3. Deleting Data

- **Command:** DELETE FROM tableName WHERE condition;
- **Purpose:** To remove records from a table.

Example

```
DELETE FROM Students  
WHERE StudentID = 1;
```

2. Querying Data:

2.1. Basic SELECT

- **Command:** SELECT column1, column2, ... FROM tableName;
- **Purpose:** Retrieves data from a table.
- Example

```
SELECT productName, amount FROM Sales;
```

2.2. SELECT DISTINCT

- **Command:** SELECT DISTINCT *column1, column2, ...* FROM *tableName*;
- **Purpose:** Return only distinct (different) values from a table. **Example**

```
SELECT DISTINCT payment_status  
FROM Sales;
```

2.3. Filter Data

- **Command:** SELECT ... FROM ... WHERE *condition*;
- **Purpose:** Retrieves data based on specific conditions.

Example:

```
SELECT productName, amount FROM Sales WHERE isCompleted = 1;
```

2.4. NOT operator

- **Command:** SELECT ... FROM ... WHERE NOT *condition*;
- **Purpose:** NOT is a logical operator in SQL that you can put before any conditional statement to select rows that do not match a specific condition.

Example:

```
SELECT * FROM Sales  
WHERE NOT payment_status = 'Completed';
```

2.5. Combining multiple conditions with AND or OR operator

- **Command:** SELECT ... FROM ... WHERE *condition1* AND/OR *condition2*;
- **Purpose:** AND is used to combine multiple conditions in a WHERE clause so that both must be true for the rows to be returned whereas OR returns a record if any one of the conditions are TRUE.

Example:

```
SELECT productName, amount FROM Sales  
WHERE isCompleted = 1 AND amount > 500;
```

2.6. Sorting Data

- **Command:** SELECT ... FROM ... ORDER BY *column* ASC|DESC;
- **Purpose:** Orders the result set based on specified columns.
- **The ORDER BY keyword sorts the records in ascending order by default.**

Example:

```
SELECT productName, amount FROM Sales ORDER BY saleDate DESC;
```

2.7. Filtering Ranges with BETWEEN

- **Command:** `SELECT ... FROM ... WHERE column BETWEEN value1 AND value2;`
- **Purpose:** Filter data where a column's values fall within a specific range (inclusive of the boundary values).

Example:

```
SELECT productName, saleDate FROM Sales
WHERE saleDate BETWEEN '2023-01-01' AND '2023-12-31';
```

2.8. Filter NULL Values

- NULL simply means an empty field
- **Command:** `SELECT column1, column2, ... FROM tableName WHERE columnName IS NULL;`
- **Purpose:** To check for missing or null values in a table.

Example

```
SELECT * FROM products WHERE price IS NULL;
```

2.9. TOP N Analysis

- **Command:** `SELECT TOP number | percent columnName(s) FROM tableName WHERE condition;`
- **Purpose:** To specify the number of records to return. **Example**

```
SELECT TOP 5 * FROM Sales
ORDER BY amount DESC
```

2.10. Aliases

- **Command:** `SELECT columnName AS alias_name FROM tableName;` (for Column)
`SELECT columnName(s) FROM tableName AS alias_name;` (for Table)
- **Purpose:** To give a table, or a column in a table, a temporary name. It only exists for the duration of that query.

Example

```
SELECT contactEmail AS Customer_Email
FROM Customers
```

2.11. Operators and Pattern Matching

I. IN Operator

- **Command:** `SELECT columnName(s) FROM tableName`
`WHERE columnName IN (value1, value2, ...);`
- **Purpose:** To match a column's value against a list of possible values, used as a shorthand for multiple OR conditions.

Example

```
SELECT * FROM Products WHERE customerId IN (3, 7, 9)
```

II. Comparison Operators (>, <, >=, <=, =, <>)

- **Command:** `SELECT columnName(s) FROM tableName`
`WHERE columnName operator value`
- **Purpose:** To compare values in a column.

Example

```
SELECT productName, amount FROM Sales  
WHERE amount >= 1000;
```

III. LIKE Operator

- **Command:** `SELECT columnName(s) FROM tableName WHERE columnN LIKE pattern;`
- **Purpose:** To search for a specified pattern in a column.

Example

```
SELECT * FROM Customers  
WHERE customerName LIKE 'A%';
```

Explanation: Retrieves all customers whose names start with 'A'.

IV. Wildcard Search

- **Purpose:** Used with LIKE operator to match unknown values in strings using some special characters

Example

```
SELECT * FROM Customers  
WHERE customerName LIKE '_t%';
```

Explanation: `_` matches any single character, and `%` matches any number of characters, so this retrieves names with 't' as the second character.

Like Operator	Description
WHERE <i>columnName</i> LIKE 'a%'	Find any value that starts with "a".
WHERE <i>columnName</i> LIKE '%a'	Find any value that ends with "a".
WHERE <i>columnName</i> LIKE '%or%'	Find any value that has "or" in any position.
WHERE <i>columnName</i> LIKE '_a%'	Find any value that starts with any character but shall have second character as 'a'.
WHERE <i>columnName</i> LIKE 'a__%'	Find any value that starts with "a" but shall have at least 3 characters in length.
WHERE <i>columnName</i> LIKE 'a%o'	Find any value that starts with "a" and ends with "o".